

Concurrency Control, Transaction Boundaries & Consistency

1. Where inventory consistency is guaranteed

Exactly one place: the `inventory` row of the product inside the Inventory DB (PostgreSQL primary), through a single atomic conditional `UPDATE ... WHERE available_quantity >= :q`, backed by `CHECK` constraints (`available >= 0`, `available + reserved + sold = total`). Redis is only a performance filter. Every other service is eventually consistent with it.

2. What happens when two requests hit at the same time (last unit)

- Both pass the Redis gate only if Redis counter allowed it (Lua is atomic: only one gets the last token).
- Even if both reached the DB, `UPDATE ... WHERE available_quantity >= 1` takes a row lock. Request A updates 1→0. Request B waits for the lock, then re-evaluates `available_quantity >= 1` on the new value (READ COMMITTED re-check) → false → 0 rows affected → `OUT_OF_STOCK`.
- The result is deterministic: one winner, one loser, and `available` never becomes negative (CHECK constraint is a last defence).

3. Approaches compared

Criterion	A. Pessimistic lock (SELECT FOR UPDATE)	B. Optimistic (version column / CAS)	C. Atomic conditional UPDATE + Redis gate (selected)	D. Queue per product (single writer)
Correctness	Strong	Strong	Strong	Strong
Behaviour under 10k contention on one row	Long lock waits, connection pile-up	Massive retry storm, most attempts fail and retry	Short row lock (one statement), losers rejected early	Smooth, serialised
DB load	High	Very high (retries)	Low (gate removes ~99% of attempts)	Low
Latency	Poor tail	Poor under contention	Good	Higher (queue hop)
Complexity	Low	Medium	Medium	High
Failure mode	Deadlock risk, lock timeouts	Livelock	Redis drift (self-healing)	Queue consumer lag

Selected: C with admission control in front. Reasons: the critical section is one statement (microseconds), no read-modify-write in application code, losers are rejected without touching the DB, and correctness never depends on Redis. Optimistic locking (`version`) is retained for *non-hot* updates (e.g. admin edits, normal SKUs) and for audit. Option D is described as the escalation path if one SKU exceeds a single DB row's throughput (see scalability doc: stock bucketing).

4. Transaction boundaries

Operation	Transaction scope (single local ACID transaction)	Isolation
Reserve	inventory UPDATE + reservation INSERT + outbox INSERT	READ COMMITTED (row lock + re-check)
Release/Expire	reservation state UPDATE (guarded) + inventory UPDATE + outbox INSERT	READ COMMITTED
Confirm / Sold	reservation UPDATE + inventory UPDATE + outbox	READ COMMITTED
Payment state change	payment UPDATE (guarded by expected state) + outbox INSERT	READ COMMITTED
Order create	processed_event INSERT + order + items + history + outbox	READ COMMITTED

No distributed transaction (2PC) across services: cross-service workflows use **saga + outbox + idempotent consumers**.

5. Consistency model

Data	Model
Inventory counts, reservation state	Strong (single primary, synchronous standby)
Payment state	Strong inside Payment DB; eventual vs gateway (reconciled)
Order state	Strong inside Order DB; eventual w.r.t. payment (seconds, bounded by recovery)
Displayed stock/"sold out"	Eventual (≤ 2 s)
Cart	Eventual / best effort
Notifications	At-least-once, deduped by <code>dedupe_key</code>

6. Redis stock gate (fast path)

Lua script (atomic):

```
-- KEYS[1]=stock:{product} KEYS[2]=idem:{customer}:{key}
if redis.call('EXISTS', KEYS[2]) == 1 then return 2 end -- duplicate in flight
local s = tonumber(redis.call('GET', KEYS[1]) or '0')
if s < tonumber(ARGV[1]) then return 0 end -- sold out
redis.call('DECRBY', KEYS[1], ARGV[1])
redis.call('SET', KEYS[2], '1', 'EX', 600)
return 1
```

- Counter is initialised from DB (`available_quantity`) when the sale opens.
- If DB step fails or returns 0 rows → `INCRBY` to give the token back.
- Release/expiry → `INCRBY` after DB commit.
- A drift detector compares Redis vs DB every 10 s and rewrites Redis from DB. **Redis can only err on the safe side if it is rebuilt from DB `available_quantity` (never from a guess).**
- If Redis is down: the gate is bypassed and the admission rate is reduced; DB still guarantees correctness.

6b. Idempotency design

Layer	Mechanism
Client	Generates Idempotency-Key (UUID) per Buy Now intent, reused on retries
Gateway	Rejects requests without key; short-lived dedupe in Redis
Reservation	<code>UNIQUE(customer_id, idempotency_key)</code> ; replay returns stored reservation
Payment	<code>UNIQUE(idempotency_key)</code> , <code>UNIQUE(reservation_id)</code> , and <code>merchant_ref</code> sent to gateway
Order	<code>UNIQUE(reservation_id)</code> , <code>processed_event(consumer, event_id)</code>
Notification	<code>UNIQUE(dedupe_key)</code>

7. Audit information

Every state change appends to `order_status_history` / `audit_log` and emits an outbox event with `event_id` , `aggregate_id` , `occurred_at` , `actor` , `correlation_id` (trace id).